View:

A view is a virtual table created by taking a subset of columns from one or more tables. A view is a logical table, as view does not contain data itself. The tables using which we create a view are called **base tables**. An output of query can be stored as a view.

Using views, we can hide information and provide security to the database.

Types of Views:

Views are of two types.

- Simple views
- Complex views.

Simple views:

- Views defined on single table.
- Can perform DML operations directly against simple views.
- Data on view's base table are changed due to the DML operations performed on the view.

Syntax:

CREATE view AS SELECT statement;

Example:

To create a view on employee table .

CREATE VIEW v_emp AS SELECT * FROM employee;

CREATE VIEW v_dept AS SELECT ename, sal*12 annual_sal FROM employee WHERE dep_name= 'HR';

Complex views:

Complex views are constructed on more than one base tables.

Complex views can contain:

- join conditions
- group by clause
- order by clause

Cannot perform DML operations against complex views directly.

We have to use instead of triggers for DML operations.

Example:

Employee table and Department table has the following structures:

Department (deptno,dname,location);

- where deptno is primary key.

Employee (empno,ename,job,deptno);

- where empno is the primary key and deptno is the foreign key.

To create a view by selecting the empno,ename and job from employee table and deptno, dname and location from department table.

CREATE VIEW v_complex1 AS SELECT emp.empno, emp.ename, emp.job, emp.deptno, dept.dname, dept.loc FROM employee, department WHERE emp.deptno=dept.deptno;

Additional View Types:

Read-only views:

Users can only use SELECT and DESCRIBE statements against these views.

DML operations are not allowed .

Syntax:

CREATE VIEW AS SELECT statement WITH READ ONLY;

Example:

CREATE VIEW v_clerk AS SELECT empno, ename, deptno, job FROM employee WHERE job = 'CLERK'

WITH READ ONLY;

With CHECK Option:

View created WITH CHECK OPTION clause specifies the level of checking to be done when doing DML against the view. View with check option ensure the view's consistency.

Syntax:

CREATE VIEW AS SELECT colums [WHERE condition] WITH CHECK OPTION;

Example :

Structure of the employee table is as follows:

Employee (eno,ename,salary,deptno);

Following statements creates a view emp_view on employee table by selecting ename,salary ,deptno from employee table for the records with deptno value 20.

CREATE VIEW emp_view AS SELECT ename, salary, deptno FROM employee WHERE deptno = 20;

Now if we use the following UPDATE statement on this view,

UPDATE emp_view SET deptno = 10;

We will be able to update deptno value for the records in the view to 10 contradicting the view definition.

We can avoid this type of situations, by creating view with check option.

CREATE VIEW emp_view AS SELECT ename, sal, deptno FROM emp2 WHERE deptno = 20

WITH CHECK OPTION;

Now, an UPDATE operation on the view with throw error.

UPDATE emp_view SET deptno = 10;

ORA-01402: view WITH CHECK OPTION where-clause violation

Force view:

Force view is the view created without having the base table.

FORCE keyword is used to create a force view.

Syntax :

CREATE FORCE VIEW AS SELECT colums [WHERE condition];

Example:

The following view will be created even employee table not exists in our schema.

CREATE FORCE VIEW v_force_view AS SELECT name ,job , deptno FROM employee

GROUP BY deptno;

In the above case, the view v_force_view will be created even employee table does not exist in schema.

Drop View:

Syntax:

DROP VIEW view_name;

Example:

DROP view emp_view;

Update View:

We can modify the definition of an Oracle VIEW without dropping it by using the Oracle CREATE OR REPLACE VIEW Statement.

Syntax :

CREATE OR REPLACE VIEW AS SELECT columns FROM table WHERE conditions;

AUTO_INCREMENT(Sequences):

In MySQL we can create a column that contains a sequence of numbers by using the AUTO_INCREMENT attribute.

The AUTO_INCREMENT attribute is helps us to create a unique number to act as a primary key in a table.

Syntax

CREATE TABLE table_name (column1 datatype NOT NULL AUTO_INCREMENT, column2 datatype [NULL | NOT NULL], ...);

AUTO_INCREMENT - The attribute used to assign a sequence of numbers automatically to a field (in essence, creating an autonumber field).

NULL or NOT NULL - Each column should be defined as NULL or NOT NULL. If this parameter is omitted, the database assumes NULL as the default.

Note : We can use the function LAST_INSERT_ID to find last value assigned by the AUTO_INCREMENT field.

Example:

```
CREATE TABLE Persons
( Social_id INT PRIMARY KEY AUTO_INCREMENT,
  last_name VARCHAR(30) NOT NULL,
  first_name VARCHAR(25),
  birthday DATE,
);
```

Note : Auto-increment can be set only on the Primary Key of a table.

Set AUTO_INCREMENT starting value:

The default AUTO_INCREMENT start value is 1.

After creating the table using the AUTO_INCREMENT attribute, we can change the starting value for the AUTO_INCREMENT field.

To change the starting value of AUTO_INCREMENT attribute, we can use ALTER TABLE statement .

Syntax:

ALTER TABLE table_name AUTO_INCREMENT = start_value;

table_name - The name of the table whose AUTO_INCREMENT value you wish to change. Since a table in

MySQL can only contain one AUTO_INCREMENT column, we need to specify the table name that

contains the sequence and do not need to specify the name of the column that contains the AUTO_INCREMENT value.

start_value -The next value in the sequence to assign in the AUTO_INCREMENT column.

Example

```
ALTER TABLE Persons AUTO_INCREMENT = 50;
```

This MySQL AUTO_INCREMENT example would change the next value in the AUTO_INCREMENT field (ie: next value in the sequence) to 50 for the Social_id field in the Persons table.